

Knowledge-Graph-Driven Data Synthesis for Low-Resource Software Development: A HarmonyOS Case Study

MINGWEI LIU, Sun Yat-sen University, China and Zhuhai Key Laboratory of Trusted Large Language Models, China

ZHENG PEI, Sun Yat-sen University, China and Zhuhai Key Laboratory of Trusted Large Language Models, China

YANLIN WANG*, Sun Yat-sen University, China and Zhuhai Key Laboratory of Trusted Large Language Models, China

ZIHAO WANG, Sun Yat-sen University, China and Zhuhai Key Laboratory of Trusted Large Language Models, China

ZIKANG LI, Sun Yat-sen University, China and Zhuhai Key Laboratory of Trusted Large Language Models, China

ENCI LIN, Sun Yat-sen University, China and Zhuhai Key Laboratory of Trusted Large Language Models, China

XIN PENG, Fudan University, China

ZIBIN ZHENG, Sun Yat-sen University, China and Zhuhai Key Laboratory of Trusted Large Language Models, China

In low-resource framework development (e.g., HarmonyOS), large language models (LLMs) often lack sufficient pre-training exposure, resulting in poor code generation performance. Although they generally preserve programming logic across languages, they frequently fail on framework-specific APIs and syntax, revealing a gap between learned algorithmic knowledge and unfamiliar framework conventions. Consequently, even advanced models such as GPT-4o struggle to produce correct code without prior exposure.

Inspired by these challenges, we propose APIKG4Syn, a framework that leverages API knowledge graphs to synthesize API-oriented question-code pairs without requiring executable environments. It incorporates both single-API and multi-API information, with the latter guided by uncertainty estimation (UE) and Monte Carlo Tree Search (MCTS), to construct high-quality fine-tuning data. For evaluation, we select HarmonyOS as a case study due to its accessible documentation and growing ecosystem, and build the first benchmark for its code generation. Experimental results show that fine-tuning Qwen2.5-Coder-7B with APIKG4Syn achieves a pass@1 of 25.00%, outperforming untuned GPT-4o (17.59%). We further observe that larger volumes of data generated by APIKG4Syn consistently lead to better fine-tuning performance, and that the optimal Single-API to Multi-API ratio is 8:2. Ablation studies also confirm the necessity and effectiveness of each component in our framework. These findings highlight the effectiveness of API-oriented data in enhancing LLM performance for low-resource software development scenarios.

CCS Concepts: • **Software and its engineering** → **Automatic programming**.

*Yanlin Wang is the corresponding author.

Mingwei Liu, Zheng Pei, Yanlin Wang, Zihao Wang, Zikang Li, Enci Lin, and Zibin Zheng are with the School of Software Engineering and the Zhuhai Key Laboratory of Trusted Large Language Models, Sun Yat-sen University, Zhuhai, China. Email: liumw26@mail.sysu.edu.cn, peizh3@mail2.sysu.edu.cn, wangylin36@mail.sysu.edu.cn, wangzh778@mail2.sysu.edu.cn, lizk26@mail2.sysu.edu.cn, linenc@mail2.sysu.edu.cn, zhizibin@mail.sysu.edu.cn. Xin Peng is with Fudan University, Shanghai, China. Email: pengxin@fudan.edu.cn.



This work is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by-nc-nd/4.0/).

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE126

<https://doi.org/10.1145/3808133>

Additional Key Words and Phrases: HarmonyOS, Domain Code Generation, Data Synthesis, API Knowledge Graph

ACM Reference Format:

Mingwei Liu, Zheng Pei, Yanlin Wang, Zihao Wang, Zikang Li, Enci Lin, Xin Peng, and Zibin Zheng. 2026. Knowledge-Graph-Driven Data Synthesis for Low-Resource Software Development: A HarmonyOS Case Study. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE126 (July 2026), 23 pages. <https://doi.org/10.1145/3808133>

1 Introduction

LLMs have achieved remarkable success in software development tasks such as code completion, generation, and comprehension [5, 16, 20, 23, 44]. However, in emerging frameworks or domain-specific environments, their performance often degrades due to limited exposure during pretraining [48]. Nonetheless, low-resource scenarios—including private libraries, proprietary frameworks, and other settings with limited data—are commonly encountered in industrial practice. HarmonyOS, a relatively new operating system with distinctive APIs, exemplifies such a low-resource scenario. Developers working with HarmonyOS face unique challenges due to complex API structures and sparse training data, which restrict the model’s ability to generate accurate and contextually appropriate code [56]. As illustrated in Figure 1, this example shows the code generated by the GPT-4o model for a breadth-first traversal task in the HarmonyOS. Although the generated code is logically correct, it omits the required API import statements, employs the disallowed any type in HarmonyOS, and calls methods that do not exist in ArrayList. **This observation indicates that the model has internalized general algorithmic knowledge during pretraining, yet it lacks domain-specific knowledge for low-resource scenarios such as HarmonyOS.**

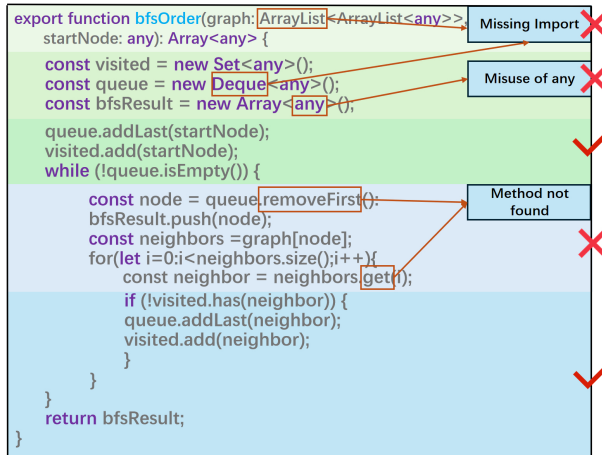


Fig. 1. The breadth-first traversal algorithm code under HarmonyOS generated by GPT-4o

For low-resource programming languages, LLMs often lack familiarity with their syntax and APIs. In contrast, they typically demonstrate stronger competence in code logic. Pan [35] et al. observed that in code translation tasks, when two languages are used to solve the same problem, models tend to preserve logical components such as control flow and algorithmic structures, while frequently introducing errors in API mapping and syntactic details. This is because, across languages such as C, Python, and Java, the solutions to a given problem are generally consistent in logic, and language-specific features rarely invalidate the overall problem-solving approach. Similarly, Alessandro Giagnorio [11] et al. reported that models perform poorly in API mapping across

programming frameworks, further indicating that LLMs exhibit uneven familiarity with APIs across different languages, although their understanding of code logic remains relatively clear.

Although domain-specific fine-tuning improves model performance, obtaining high-quality datasets for new platforms is still difficult. Existing methods, like OSS-Instruct [49], generate training data for supervised fine-tuning (SFT), but often lack complete API semantic coverage, sufficient multi-API reasoning, or rely heavily on real usage examples that are not available for new frameworks. These limitations create gaps in data quality and applicability for low-resource scenarios.

To address these challenges, **we propose APIKG4Syn, a knowledge-graph-based data synthesis framework specifically designed for domain-specific software development.** Our approach constructs a structured API knowledge graph from documentation, capturing function descriptions, parameters, return values, and hierarchical relationships. LLMs are then leveraged to quantify the familiarity with different API nodes, which in turn guides Monte Carlo Tree Search [33] (MCTS) to identify unfamiliar or highly correlated API nodes for generating multi-API joint scenarios. For single-APIs, we also provide representative single-API application scenarios. Based on this information, API-Oriented question-code tuples are synthesized to enrich LLMs' understanding of APIs in low-resource environments, and even to enable the construction of corresponding models in zero-resource scenarios. In contrast to approaches that rely on high-quality exemplar code [49], APIKG4Syn leverages LLMs with API documentation to generate realistic code without referring to real-world cases, even in unfamiliar ecosystems. This enables scalable data construction while explicitly promoting API reuse to avoid redundancy. The resulting data effectively supports fine-tuning for low-resource languages.

In addition, we construct a HarmonyOS code generation benchmark to quantitatively evaluate LLM performance in this low-resource scenario, providing a standard testbed for future research. We conduct extensive experiments (details in Section 3) to evaluate the effectiveness of our method. The results demonstrate that fine-tuning LLMs with our synthesized data yields substantial improvements over baseline methods on both single-API and multi-API tasks, without relying on any existing real-world cases. The data ratio and data volume studies further verify the correctness of the performance trends of models fine-tuned with data generated by APIKG4Syn, as well as the effectiveness of our selected data ratio. Moreover, ablation studies confirm the necessity of each component in our framework, and the fine-tuned models consistently outperform the plain text + RAG approach for code generation. Specifically, our approach attains a pass@1 of 25.00% on HarmonyOS benchmark, exceeding the strongest baseline (10.19%) by 14.81%. These findings validate the efficiency and effectiveness of API knowledge graph-guided data synthesis.

In summary, this paper makes the following contributions:

- We propose a knowledge-graph-driven data synthesis framework that generates targeted training data without relying on execution examples.
- We construct a HarmonyOS code generation benchmark to provide a standard evaluation testbed for future research.
- We validate the effectiveness of our method through quantitative experiments, showing improvements in LLM performance for both single-API and multi-API scenarios.
- We also release the OHBen Dataset, our generated training dataset, as an open-source contribution to support fine-tuning of HarmonyOS-related LLMs.

2 Method

LLMs often lack sufficient exposure to APIs in emerging frameworks, leading to poor generalization in low-resource settings. To address this gap, we design **APIKG4Syn**, a knowledge-graph-driven

data synthesis framework that creates realistic, API-grounded training samples for HarmonyOS. The approach systematically constructs an API knowledge graph, generates single-API questions with factual grounding, and extends to multi-API scenarios using Monte Carlo Tree Search (MCTS) [33].

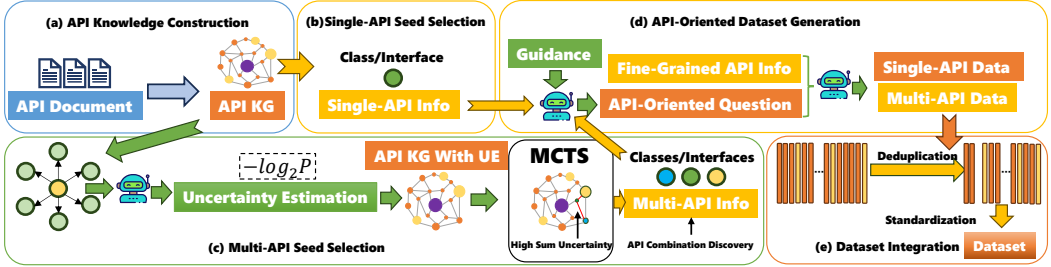


Fig. 2. Approach Overview of APIKG4Syn

Figure 2 illustrates the overall workflow. The framework operates through five key stages:

- (1) **API Knowledge Graph Construction:** HarmonyOS API documentation is parsed to extract modules, classes, methods, and properties, along with hierarchical and semantic relationships. Each node is enriched with metadata such as functional description, parameters, and return values.
- (2) **Single-API Seed Selection:** By leveraging the API knowledge graph, the system retrieves and provides the corresponding single-API information based on a specific usage scenario.
- (3) **Multi-API Seed Selection:** Leveraging the API knowledge graph, we employ LLMs to compute UE, which guides a Monte Carlo Tree Search (MCTS [33]) to identify node paths with the highest cumulative uncertainty. The relevant nodes along these paths are then extracted and combined to construct multi-API information.
- (4) **API-Oriented Question-Code Tuples Dataset Construction:** Using the provided single-API and multi-API information, LLMs generate appropriately tailored API-Oriented problem scenarios. The models then retrieve fine-grained API information from the API knowledge graph to produce corresponding API-Oriented code solutions, thereby constructing an API-Oriented question-code tuple dataset.
- (5) **Dataset Integration and Post-processing:** The single-API and multi-API generated API-Oriented data are combined into a unified dataset. Post-processing steps include deduplication, formatting, and validation of questions and answers, ensuring consistency and quality for downstream fine-tuning tasks.

These five steps form the core of the APIKG4Syn methodology. Details regarding implementation, dataset construction, and descriptive statistics are provided in Section 2.6.

2.1 API Knowledge Graph Construction

In this stage, we parse the API documentation of low-resource frameworks (HarmonyOS) to build a structured knowledge graph representing the API ecosystem. The graph captures hierarchical relationships among modules, classes, methods, and properties. Each node includes metadata such as functional descriptions, parameters, and return values. This structured representation forms the basis for generating both single-API and multi-API training data. The construction process involves:

- Extract code information and text information from the API documentation.
- Transform restructured code and semantic information into a professional API knowledge graph.

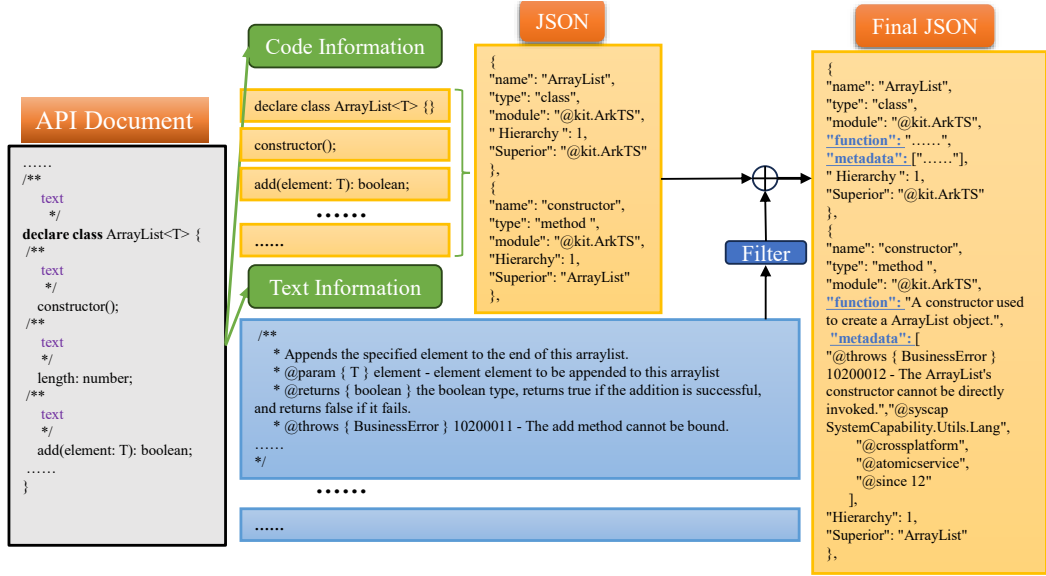


Fig. 3. API Documentation Conversion Workflow

Data Reorganization. As illustrated in Figure 3, we extracted all API documentation from the SDK folders of DevEco Studio, the official IDE for HarmonyOS. We scanned the entire set of API documents and applied rule-based matching to extract information from each line. The extracted data typically comprise two categories: code-related information and textual information.

Code Info. The code-related information contains all code snippets from the API documentation, which inherently encode the graph structure of the APIs. By analyzing these code snippets, we can determine the hierarchical organization of different methods, properties, classes, interfaces, namespaces, and enums.

Text Info. The textual information includes metadata for the corresponding APIs, such as functional descriptions, parameters, and return values. This information can be leveraged by large language models as reliable evidence for subsequent code generation.

API Knowledge Graph Construction. All entities are extracted from the code and their relationships are formed based on nesting structures to create an initial graph. The corresponding textual information is parsed, with rule-based matching and filtering to get the most up-to-date and valid semantic data. By combining this semantic data with the initial graph structure, as shown in Figure 4, an API knowledge graph is created.

2.2 Single-API Seed Selection

At this stage, the APIKG4SYN framework traverses all non-leaf nodes in the knowledge graph, such as classes and interfaces, and retrieves the semantic and code information of these nodes along with their child nodes for output. The selection of single-API seeds is illustrated in Figure 5.

2.3 Multi-API Seed Selection

At this stage, for multi-API information, we first employ a large language model to compute the UE for each non-leaf node, thereby quantifying the model's unfamiliarity with the node. This UE then guides a Monte Carlo Tree Search (MCTS) [33] to identify the top-5 highest-scoring paths. From each path, which contains several high-scoring nodes, 2–3 nodes are randomly sampled and

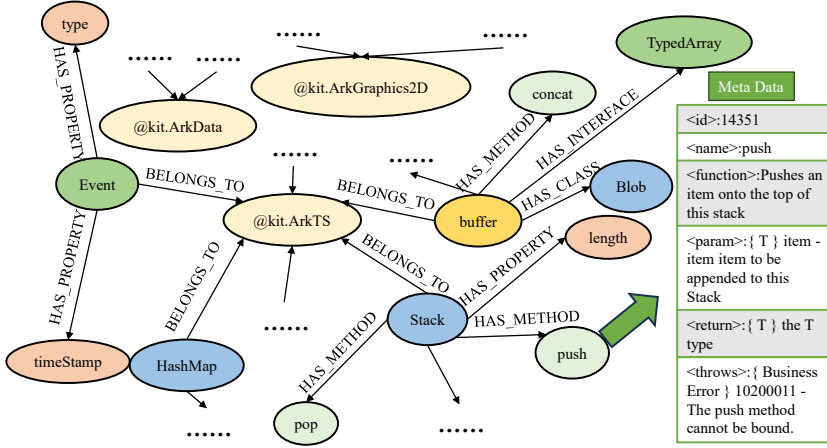


Fig. 4. API Knowledge Graph and Node Metadata

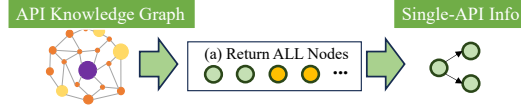


Fig. 5. Single-API Non-Leaf Node Selection and Data Retrieval

combined to construct multi-API information. The detailed procedure for synthesizing multi-API information is as follows:

- Compute the UE for each non-leaf node.
- MCTS [33] is executed with the UE value serving as the reward function.

Uncertainty Estimation Construction. Shannon (1948) proposed that uncertainty estimation [39] quantifies the information content of a "fact" under a given probability distribution. In our API knowledge graph, a "fact" is represented by a non-leaf node, its code and semantic information, and its child nodes. This fact is formalized as a triplet $\tau = \langle u, \rho, v \rangle$. To measure how well a large language model understands this information, we transform the triplets into a form for computing conditional probabilities, specifically $P(v \mid u, \rho)$. Given a head entity u and a predicate ρ , we estimate the probability that a tail entity v belongs to u . The negative logarithm of this probability gives the uncertainty estimation, calculated using the following formula:

$$I(u, \rho, v) = -\log_2 P(v \mid u, \rho) \quad (1)$$

Let u denote a non-leaf node, such as the *ArrayList* class, and v denote one of its child nodes, such as the *add* method. The relation ρ represents the membership of v under u , and in our knowledge graph, ρ is modeled as a directed relationship. $P(v \mid u, \rho)$ thus represents the probability of v belonging to u given the leaf node, while $I(u, \rho, v)$ denotes the computed uncertainty estimation. This value reflects the extent to which a large language model understands the internal structure of the upper-level API, providing a quantifiable measure that can be used in further analyses. Finally, the computed uncertainty estimation value is stored as a property of the corresponding node.

MCTS. The overall workflow of integrating the MCTS [33] algorithm into our APIKG4Syn framework is illustrated in Figure 6.

Node Selection. Starting from a root node, the algorithm recursively selects child nodes according to the chosen selection strategy until a leaf node or an unexpanded node is reached. In our

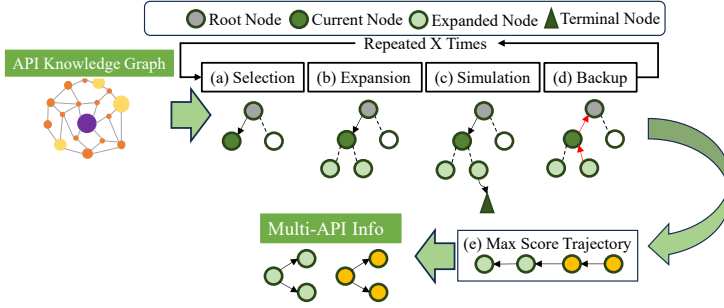


Fig. 6. Multi-API Non-Leaf Node Selection and Data Retrieval

implementation, we adopt the Upper Confidence Bound (UCB1 [1]) strategy. Importantly, the root node is not fixed; instead, each node is treated as a root once, aiming to explore a broader range of API combination paths. The UCB1 [1] strategy is defined as follows:

$$UCB1(n) = \begin{cases} +\infty, & N(n) = 0, \\ Q(n) + c \cdot \sqrt{\frac{\ln N_{\text{total}}}{N(n)}}, & N(n) > 0 \end{cases} \quad (2)$$

$$Q(n) = \frac{W(n)}{N(n)} \quad (3)$$

In the formula, n is the current node, $N(n)$ is the visit count of node n , and $W(n)$ is its total accumulated reward. In our implementation, the reward for each node is the uncertainty estimation from Section 2.1. $Q(n) = W(n)/N(n)$ represents the average reward of the node. The parameter c is the exploration coefficient, balancing exploration and exploitation. N_{total} is the total visit count of the parent node or the sum of all sibling nodes' visits. The value $+\infty$ ensures unvisited nodes are selected at least once.

Expansion. In the expansion step, if the selected leaf node has unvisited neighbors, one is randomly chosen and added as a new child node, expanding the tree and exploring new states.

Simulation. After expansion, a simulation starts from the newly added node. The algorithm randomly selects unvisited successor nodes until a terminal node or a leaf node with no unvisited successors is reached. The simulated path is denoted as:

$$\pi = (n_0, n_1, \dots, n_T), \quad (4)$$

where n_0 is the newly expanded node and n_T is the terminal node. The cumulative reward of this path is calculated as:

$$R(\pi) = \sum_{t=0}^T r(n_t), \quad (5)$$

where $r(n_t)$ represents the reward of node n_t . In our implementation, the reward for each node is defined as the uncertainty estimation introduced in Section 2.1. This simulation step provides an estimate of the potential value of the newly expanded node, guiding subsequent backpropagation and selection.

Backup. After simulation, the reward obtained from the simulated path is propagated back along the nodes traversed during selection and expansion. For each node n in the path $\pi = (n_0, n_1, \dots, n_T)$,

the visit count and total accumulated reward are updated as follows:

$$N(n) \leftarrow N(n) + 1, \quad W(n) \leftarrow W(n) + R(\pi), \quad Q(n) = \frac{W(n)}{N(n)}, \quad (6)$$

where $R(\pi)$ is the cumulative reward of the simulated path. This backpropagation step refines the estimated value of each node, influencing future selection decisions and guiding the search towards nodes with higher expected rewards.

Max Score Trajectory. After repeating the process multiple times, we obtain several paths. We select the top five highest-scoring paths, each containing multiple non-leaf API nodes. From these nodes, we randomly select two to three API nodes, and return their corresponding code and semantic information. This procedure provides multi-API nodes along with their code and semantic data.

2.4 API-Oriented Question-Code Tuples Generation

This stage focuses on the generation of an API-Oriented question-code tuple dataset. Specifically, we leverage the code information and semantic descriptions of API nodes retrieved from the API knowledge graph. By utilizing these reliable sources in combination with carefully designed prompts, the system generates well-structured, low-resource framework-related questions. Subsequently, finer-grained API information is retrieved based on these questions and provided to LLMs, which then produce the corresponding code solutions. The key steps include:

- **Target API Information Retrieval:** Select non-leaf nodes as targets and provide reliable code information and semantic information.
- **API-Oriented Questions Generation:** LLMs are employed to generate well-structured low-resource framework-related questions grounded in the provided API information.
- **API-Oriented Code Generation:** LLMs address low-resource framework-related questions by leveraging the questions themselves together with fine-grained API information retrieved from the API knowledge graph.

Prompting For Generating Question. For both single-API and multi-API data generation, the previous step provides the code and semantic information of the target nodes, as well as the code and semantic information of their corresponding child APIs. With this information, we populate a specially designed prompt for question generation. This prompt is tailored to generate programming questions in the low-resource framework environment and is capable of producing questions involving single-API as well as multi-API usage, as illustrated in Figure 7.

Prompting For Generating Code. Once low-resource framework-related questions are formulated, LLMs are employed to solve them. To enhance problem-solving, the APIKG4Syn approach retrieves fine-grained code and semantic information from the knowledge graph for the relevant single-API or multi-API elements mentioned in each question. This retrieved information is incorporated into a carefully designed code-generation prompt, as illustrated in Figure 7. By leveraging this enriched knowledge, the model gains a comprehensive understanding of how each API is used and the functionalities they provide, thereby enabling more effective solutions to low-resource framework-related problems.

2.5 Dataset Integration and Post-processing

Dataset integration and post-processing occur after the generation of both single-API and multi-API data. In total, our method produces 6,400 single-API data points and 1,600 multi-API data points. The generated API-oriented questions of different types are shown in Figure 8.

The generated datasets are then merged and subjected to post-processing to ensure consistency and quality. This step includes:

You are given the definition of an/multiple <code>{framework}</code> API(s), including its methods, properties, and documentation. Instruction		You are an expert <code>{framework}</code> developer and need to implement code for the following question: Instruction	
<code>[API]: {API_name}'n</code> <code>[API Members]: {API_member}</code> <code>API Info</code>		Question: <code>{question}</code> Here is the API definition you should use: <code>[API]: {API_name}</code> <code>[Function]: {function},</code> <code>[Meta data]: {meta_data}</code> <code>[Relevant API details]: {corpus}</code> Question & API Details	
Your task: Design **{num} simple programming exercises** based on this/these API. [Guidelines]: 1. For each exercise, first **reason step by step** about how it can be solved using both the **sub-API methods/properties** and the **parent API methods/properties** . 2. Each exercise must **explicitly specify the exact API methods/properties that must be used to solve the problem** (e.g., "use the 'sort' and 'remove' methods"). 3. Each exercise must combine **at least two different methods/properties** . 4. Output only the **final problem statements** , not the reasoning process. Requirements		Please write the implementation code. Return only the code without any explanations. [Requirements]: 1. The code must use only the provided sub-methods and properties of these APIs, and the use of any other methods or properties is strictly prohibited. 2. All functions, classes, or implementations must be marked with export. 3. Usage of 'any' or 'unknown' types is prohibited. 4. The first line must include: <code>import {{API_name}}</code> from ' <code>{module}</code> '; 5. <code>{type_statement}</code> Requirements	
[Example]: problem 1: ["Write a function that takes an 'ArrayList' object and removes its maximum value **using the 'sort' and 'remove' methods** ."]	[Example]: problem 1: ["Write a function that uses an 'ArrayList's 'forEach' and a 'buffer's 'write' / 'alloc' methods to write all elements to the 'buffer'."]		
Single API Example	Multi API Example		
[Output Format]: {{ "problem": {{ ["Problem statement with explicit API usage"], ["Problem statement with explicit API usage"],...} }} }		[Output Format]: {{ "code": "Your implementation code" }}	
Output Format		Output Format	

Fig. 7. Left: Prompt for question generation. Right: Prompt for code generation.

Single-API Question	Multi-API Question
Write a function that takes a 'Blob' object and returns its size in kilobytes (KB) rounded to 2 decimal places. Use the 'size' property and perform the necessary calculations."	Create a function that first uses the 'getFirst' and 'getLast' methods of 'LinkedList' to retrieve the first and last elements of a list, then uses these elements to construct an XML event log (using the 'EventType' enum of 'xml') and stores the log in a new 'LinkedList' instance using the 'addFirst' and 'addLast' methods."

Fig. 8. Single-API Scenario Questions and Multi-API Scenario Questions

- Deduplication of questions and answers.
- Standardizing the format of dataset entries.
- Preparing the dataset for downstream training and evaluation.

Deduplication. To ensure that the generated dataset does not contain samples similar to those in the benchmark (i.e., with similar questions or solutions), we employ a fuzzy text-matching algorithm based on the Levenshtein distance to identify and remove similar samples. Equivalent numbers of new samples are then generated to maintain the dataset size, thereby preserving the rigor and consistency of the experiments.

Standardization. Our original data format does not conform to the training format required by LLaMA-Factory [58]. Therefore, we standardize the dataset into the format adopted by LLaMA-Factory [58], to facilitate subsequent fine-tuning.

2.6 Implementations

The APIKG4SYN framework is implemented in Python and leverages the DeepSeek-V3 LLM for question and answer synthesis. Knowledge graph construction, MCTS [33], and data integration are automated to produce a structured dataset in JSON format. Each record contains:

- question: a natural-language programming query.
- code: corresponding explanation or code snippet.
- api_nodes: the set of API nodes involved in generating the sample.

Model Selection.

Deepseek-V3. Deepseek-V3 [7] is selected to generate uncertainty estimation values, low-resource framework-related questions, and their corresponding code solutions. The choice of Deepseek-V3 [7] is motivated by its strong coding capabilities, superior reasoning ability, and relatively low cost, making it a suitable model within our methodological pipeline.

Fine-tuned LLMs. We select Qwen2.5-Coder-7B, Mistral-7B, and Deepseek-R1-7B as the LLMs for fine-tuning, as they are relatively lightweight in terms of parameter size while still exhibiting strong coding capabilities. In addition, Mistral-7B is a model fine-tuned with the OSS-Instruct [49] approach, which further motivated our interest in including it in our study.

LLMs used for comparison. We select Qwen3-32B, GPT-4o, o4-mini, and Deepseek-R1-671B as LLMs for comparison. Qwen3-32B is the upgraded version of Qwen2.5-Coder-7B, and we specifically choose the 32B variant to facilitate a direct comparison between the fine-tuned Qwen2.5-Coder and its successor. Deepseek-R1-671B represents the full-scale version of Deepseek-R1-7B, which further motivates our interest. In addition, we include GPT-4o and o4-mini from OpenAI: GPT-4o is known for its exceptional capabilities and stands among the leading commercial closed-source models, while o4-mini, a smaller-scale variant released by OpenAI, is chosen to assess the performance of lightweight models from the same provider.

Fine-tune configuration. For hardware, we used an RTX 4090 GPU for fine-tuning and an Intel Xeon Platinum 8352V CPU running Linux. The fine-tuning platform was LLaMA-Factory [58], a comprehensive framework for LLM fine-tuning. All models in our experiments were fine-tuned using this platform. We applied the Lora method, training for 6 epochs with the AdamW optimizer, an initial learning rate of $5e-5$, and a batch size of 2.

3 Evaluation

We conduct extensive experiments to evaluate the effectiveness of APIKG4Syn. Specifically, we aim to answer the following research questions:

- **RQ1 (Effectiveness):** Is the APIKG4Syn method more effective than data synthesized using the OSS-Instruct [49] method?
- **RQ2 (Data Scale Impact):** The scale of synthetic data and the ratio between independent API code data and linked API code data influence the fine-tuning of LLMs.
- **RQ3 (Single-API Data Vs. Multi-API Data):** How does single-API Data compare with multi-API Data in enhancing the fine-tuning effectiveness of LLMs?
- **RQ4 (Component Effectiveness):** How effective are the individual components of the APIKG4Syn method?

3.1 Experimental Setup

Datasets. In this study, we introduce the first dedicated HarmonyOS benchmark designed to assess the proficiency of LLMs in HarmonyOS programming. Considering that existing low-resource frameworks such as Qt, ROS, and HarmonyOS lack publicly available benchmark, and given the relatively high accessibility of HarmonyOS documentation, we select HarmonyOS as the target framework for evaluation. As shown in the Figure 11, we devoted approximately 300 human-hours to manually curate a comprehensive suite of tasks covering algorithms, graphics, gaming, and audio. Each task includes minimal yet essential test cases to rigorously evaluate code correctness and logical consistency. Specifically, for each sample in the benchmark, as illustrated in Figure 9,

it comprises a problem that requires a solution using HarmonyOS APIs, a corresponding code implementation, and a complete test script.

Given an integer n, return the n-th Fibonacci number. The Fibonacci sequence is defined as: $F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)$, for $n > 1$. Use <code>@kit.ArkTS.ArrayList<number></code> if needed to store intermediate results (memoization). Use <code>.add(value)</code> to append, <code>.get(index)</code> to access, and <code>.size()</code> to check the size of the list. Question	<pre>import { describe, it, expect } from '@ohos/hypium'; import { fibonacci } from './main/ets/Algorithms/fibonacci'; export default function fibonacciTest() { describe('fibonacci', () => { it('should return 0 for n=0', 0, () => { expect(fibonacci(0)).toEqual(0); }); it('should return 1 for n=1', 0, () => { expect(fibonacci(1)).toEqual(1); }); it('should return 5 for n=5', 0, () => { expect(fibonacci(5)).toEqual(5); }); it('should return 55 for n=10', 0, () => { expect(fibonacci(10)).toEqual(55); }); it('should return 832040 for n=30', 0, () => { expect(fibonacci(30)).toEqual(832040); }); }); }</pre> Test
<pre>import { ArrayList } from '@kit.ArkTS'; export function fibonacci(n: number): number { if (n === 0) return 0; if (n === 1) return 1; const memo = new ArrayList<number>(); memo.add(0); memo.add(1); for (let i = 2; i <= n; i++) { memo.add(memo[i - 1] + memo[i - 2]); } return memo[n]; }</pre> Code	

Fig. 9. An example from the OHBen

Moreover, when executing the benchmark, we employ the prompt illustrated in Figure 10, which facilitates the construction of testable code based on the unit tests to verify whether the implementation passes.

You are an expert {framework} developer. Analyze the following unit tests to identify the exact function/class/method signature or object under test. Then, based on the instruction and the tested API or object in the unit tests, implement the correct code. Instruction	
### Instruction: {inst}\n	### Unit Tests: {test} Input
Requirements: 1. The implementation must define exactly the function/class/method that is being tested. 2. Ensure the name, parameters, and return type exactly match the unit tests. 3. Return only the {framework} code, with no explanations. 4. No Markdown formatting, just plain code. Requirements	

Fig. 10. Prompt Used for Evaluating HarmonyOS Benchmark

As shown in Figure 11, when constructing the HarmonyOS benchmark, 40.7% of the samples belong to the algorithm category, which is higher than any other category. This predominance is due to the fact that algorithm-related tasks involve numerous fundamental APIs, such as **Stack**, **HashMap**, and **ArrayList**, which are essential for development. Therefore, we intentionally included a larger proportion of algorithmic tasks to evaluate models' understanding of core APIs.

To better highlight the models' capability in applying the HarmonyOS for practical application development, we also designed a substantial number of application-oriented tasks, accounting for approximately 17.6% of the benchmark. Subsequent categories include system, game, and graphics-related tasks, which reflect common scenarios in HarmonyOS development.

Baselines. We selected OSS-Instruct [49] as a baseline to compare with our proposed APIKG4Syn method. OSS-Instruct [49] is a highly effective and well-established program synthesis approach, renowned for its ability to fine-tune 7-billion-parameter models using synthetic data with strong performance. This method represents a mature and robust reference in code generation and is widely adopted in the field of code data synthesis. To the best of our knowledge, there are currently

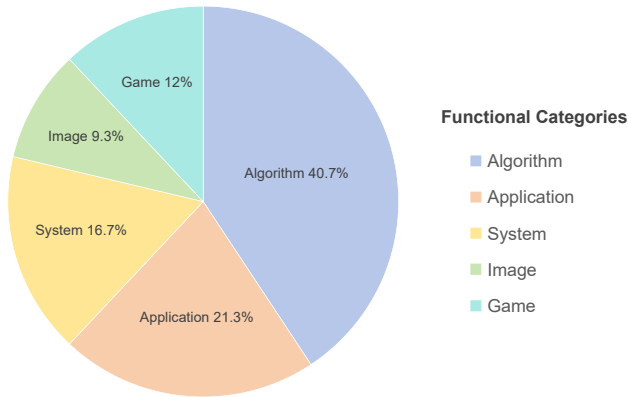


Fig. 11. Distribution of HarmonyOS benchmark sample categories

no other methods specifically leveraging code synthesis data for model fine-tuning. Therefore, we select OSS-Instruct [49] as the sole baseline for comparison.

When using OSS-Instruct [49], the method relies on a so-called heuristic dataset. However, to date, no publicly available HarmonyOS-related datasets are known. To ensure a fair comparison, we crawled 19,070 code snippets from the HarmonyOS documentation to construct a heuristic dataset, thereby adapting OSS-Instruct [49] for HarmonyOS-specific tasks.

LLM. For fine-tuning, we selected the 7B-parameter versions of Deepseek-R1-7B, Qwen2.5-Coder-7B, and Mistral-7B. The 7B scale was chosen as it represents a relatively efficient configuration that balances computational feasibility with experimental rigor. Both Deepseek-R1-7B and Qwen2.5-Coder-7B have gained recognition for their strong code generation capabilities, making them natural candidates for our experiments. In addition, Mistral-7B was included since it serves as a representative model trained with the OSS-Instruct [49] method, which we adopt as our baseline; therefore, it was selected as one of the fine-tuned models.

For LLMs without fine-tuning, we evaluated GPT-4o, o4-mini, Deepseek-R1-671B, and Qwen3-32B. GPT-4o was chosen as a powerful commercial LLM with strong reasoning abilities, while o4-mini, a lightweight model from OpenAI, also provides competitive reasoning capacity. To benchmark the fine-tuned Deepseek-R1-7B against its full-scale counterpart, we included Deepseek-R1-671B. Similarly, to assess the fine-tuned Qwen2.5-Coder-7B in comparison with a larger variant, we evaluated Qwen3-32B.

3.2 RQ1: Effectiveness

Design. Table 1 reports a comparative analysis of fine-tuning results obtained using synthetic data generated by our APIKG4Syn method and the OSS-Instruct approach [49], evaluated across a diverse set of LLMs, including both commercial and open-source variants. The models under consideration comprise Qwen-7B, Qwen2.5-7B, Qwen2.5-Coder-7B, Deepseek-LLM-7B, Deepseek-R1-7B, Mistral-7B, Deepseek-R1-671B, Qwen3-32B, GPT-4o, and o4-mini. Owing to computational resource constraints, fine-tuning experiments were conducted exclusively on the 7B-parameter versions of Qwen, Qwen2.5, Qwen2.5-Coder, Deepseek-LLM, Deepseek-R1, and Mistral. Given its consistently strong performance across a wide range of tasks, GPT-4o was selected as the representative commercial LLM. In addition, o4-mini was included as a lightweight GPT variant to assess its effectiveness on the HarmonyOS benchmark. Among open-source LLMs, Qwen3 [43] and Deepseek-R1 have demonstrated particularly competitive capabilities, and were therefore chosen as representative models in this category.

Table 1. Comparison of pass@1 performance on the HarmonyOS benchmark. Models fine-tuned with enhanced data using APIKG4SYN and OSS-Instruct are compared against their w/o SFT counterparts.

Model	Release Date	Method	pass@1
Deepseek-R1-671B	2025-01-20	w/o SFT	3.70%
GPT-4o	2024-05-13	w/o SFT	17.59%
o4-mini-7B	2025-04-16	w/o SFT	9.26%
Qwen3-32B	2025-04-29	w/o SFT	9.26%
Deepseek-LLM-7B	2023-11-29	w/o SFT	0%
		APIKG4SYN	7.41%
		w/o SFT	0%
Deepseek-R1-7B	2025-01-20	OSS-Instruct	0.93%
		APIKG4SYN	9.26%
Mistral-7B	2023-09-27	w/o SFT	4.63%
		OSS-Instruct	4.63%
		APIKG4SYN	14.81%
Qwen-7B	2023-08-04	w/o SFT	1.85%
		APIKG4SYN	5.56%
Qwen2.5-7B	2024-09-18	w/o SFT	6.48%
		APIKG4SYN	16.67%
Qwen2.5-Coder-7B	2024-11-12	w/o SFT	8.33%
		OSS-Instruct	10.19%
		APIKG4SYN	25.00%

We evaluated 7B models (Deepseek-R1-7B, Mistral-7B, Qwen2.5-Coder-7B) zero-shot on the HarmonyOS benchmark. To assess synthetic data quality, we fine-tuned these models using OSS-Instruct [49] and APIKG4SYN-generated data. Additionally, to investigate model evolution, we tested DeepSeek-LLM-7B, Qwen-7B, and Qwen2.5-7B under zero-shot and APIKG4SYN fine-tuning settings (excluding OSS-Instruct to isolate iteration effects). Finally, commercial models (GPT-4o, o4-mini, Deepseek-R1-671B) served as zero-shot references.

Results.

Comparison with the OSS-Instruct [49] Method. Using the OSS-Instruct [49] method, we scanned and extracted all HarmonyOS code blocks from the documentation in the Gitee repositories of HarmonyOS, and organized them into a heuristic library as described in the OSS-Instruct [49] framework. To evaluate the fine-tuning performance under an equal data scale, we generated 8000 HarmonyOS samples with OSS-Instruct [49]. As shown in Table 1, under the same model architecture, fine-tuning with data generated by the APIKG4SYN method yields an average improvement of 11.11% in pass@1 across three models, compared with fine-tuning on OSS-Instruct [49] data. The improvements are substantial, primarily because, given the same data volume, our synthesized data more closely adhere to API usage conventions and logical patterns. In contrast, OSS-Instruct [49] relies on high-quality prompt libraries, which are not available in low-resource environments. This, in turn, further demonstrates the superior effectiveness of our approach, and we will further analyze the results across different models in the following discussion.

Qwen2.5-Coder-7B. For Qwen2.5-Coder-7B, the pass@1 score without fine-tuning is 8.33%, which is the highest among all models prior to fine-tuning. We hypothesize that this may be due to Qwen2.5-Coder-7B having acquired preliminary exposure to HarmonyOS during pre-training,

thereby demonstrating stronger baseline performance compared to other models. When fine-tuned with OSS-Instruct [49] data, Qwen2.5-Coder-7B achieves a modest improvement of 1.86%. In contrast, fine-tuning with data generated by the APIKG4Syn method results in a substantial improvement of 15.74%, clearly outperforming OSS-Instruct [49]. After fine-tuning, the model demonstrates a deeper understanding of API information. As shown in Figure 12, for the same problem, the fine-tuned model successfully addresses issues such as using APIs without proper import statements and invoking non-existent methods. This provides evidence that fine-tuning enables the model to acquire and apply API-specific knowledge.

Qwen-7B & Qwen2.5-7B. For Qwen-7B, pass@1 improves from 1.85% to 5.56% after APIKG4Syn-based fine-tuning, indicating a modest gain. For Qwen2.5-7B, it increases from 6.48% to 16.67%, showing a more substantial improvement. However, both still lag behind Qwen2.5-Coder-7B. This gap likely stems from their earlier release stages, when HarmonyOS resources were more limited and training data quality was lower, leading to comparatively weaker coding capabilities than later versions in the same family.

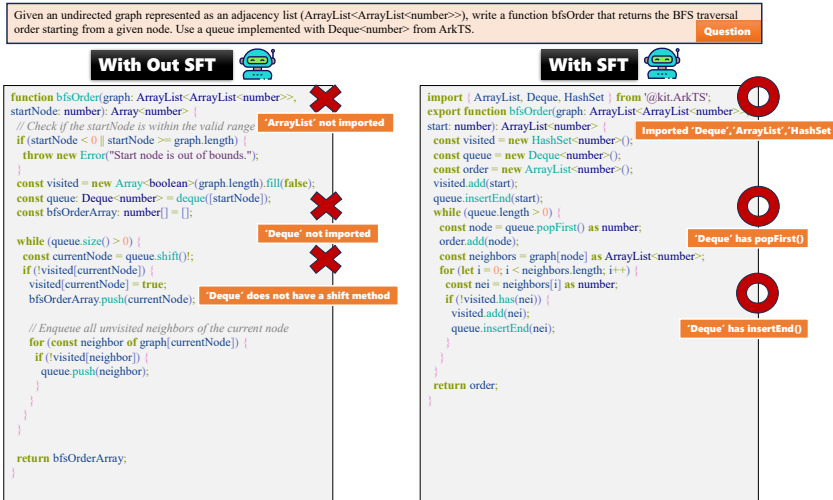


Fig. 12. A comparative example illustrating the outputs of the Qwen model on the same problem, with and without fine-tuning

Deepseek-R1-7B. An interesting observation arises with the Deepseek-R1-7B model: without fine-tuning, it fails to pass any test cases in the HarmonyOS benchmark (i.e., a pass@1 of 0). Even after fine-tuning with OSS-Instruct [49] data, its pass@1 only reaches 0.93%, indicating negligible improvement. However, when fine-tuned with APIKG4Syn-generated data, the pass@1 increases significantly to 9.26%, representing a 8.33% gain over OSS-Instruct [49].

Deepseek-LLM-7B. For DeepSeek-LLM-7B, the pass@1 score is 0% without fine-tuning and improves to 7.41% after fine-tuning with data generated by APIKG4Syn. The underlying reasons are generally consistent with those observed for Qwen-7B and Qwen2.5-7B, namely its earlier release stage and the relatively limited availability and quality of HarmonyOS-related training resources at that time, which constrain its coding capability compared to more recent model iterations.

Mistral-7B. For the Mistral-7B model, the pass@1 score remains identical (4.63%) both without fine-tuning and after fine-tuning with OSS-Instruct [49] data. We attribute this to the OSS-Instruct [49] dataset lacking sufficient API-related information, which limits the model's ability to correctly generate HarmonyOS programs. By contrast, APIKG4Syn-generated data provides

more comprehensive API information, enabling more effective fine-tuning. Consequently, the pass@1 for Mistral-7B improves to 14.81% with APIKG4Syn, demonstrating a clear advantage over OSS-Instruct [49].

Comparison with the non-fine-tuned model. Although Deepseek-R1-671B is the full-scale version, it achieves only 3.70% pass@1, likely due to limited HarmonyOS-related pretraining data. In contrast, Deepseek-R1-7B and DeepSeek-LLM-7B fine-tuned with APIKG4Syn significantly outperform it. Similarly, despite its larger scale and broader corpus, Qwen3-32B is surpassed by fine-tuned Qwen2.5-Coder-7B and Qwen2.5-7B, further validating our fine-tuning strategy. Among commercial models, GPT-4o achieves the best performance without fine-tuning (17.59% pass@1), yet still falls short of Qwen2.5-Coder-7B. Overall, fine-tuning with APIKG4Syn enables smaller models to outperform much larger counterparts.

Finding 1: In low-resource settings, model performance does not increase monotonically with scale; appropriate fine-tuning can even enable smaller models to outperform larger ones. Our APIKG4Syn method effectively enhances model capabilities under such conditions.

3.3 RQ2: Impact of Data Scale on Model Performance

We systematically investigate the impact of data scaling and the ratio of single-API to multi-API data on LLM fine-tuning by training with varying volumes and compositions of synthetic code. To minimize random variability, experiments are conducted on three distinct local models across multiple data scales. This study aims to elucidate the effects of data volume and proportion on the training performance of 7B-parameter LLMs.

Design.

Data Volume Experiment: To investigate the impact of data volume on model fine-tuning, we randomly sample datasets of sizes 1,000, 2,000, 3,000, 4,000, 5,000, 6,000, 7,000, and 8,000, while preserving the original ratio between single-API and multi-API data. This allows us to examine how different data volumes affect model performance under identical fine-tuning conditions.

Data Ratio Experiment: To investigate the influence of the ratio between single-API and multi-API data on model fine-tuning, we fix the volume of multi-API data while varying the quantity of single-API samples. Specifically, we set the single-API to multi-API ratios to 1:9, 2:8, 3:7, 4:6, 5:5, 6:4, 7:3, 8:2, and 9:1, using approximate sample counts where exact division is not possible. This allows us to systematically examine how different data compositions affect model performance.

Results.

Data Volume: As illustrated in Figure 13 and Table 3, Mistral-7B and Deepseek-R1-7B exhibit a non-monotonic trend as fine-tuning data increases, a pattern consistent with the double descent phenomenon [2] where generalization error temporarily rises near the interpolation threshold due to convergence toward sharp minima. However, as data volume continues to grow, stronger constraints steer the optimization toward flatter minima, thereby restoring performance. In sharp contrast, Qwen2.5-Coder-7B bypasses this volatility; benefiting from robust pretrained representations and favorable initialization, it maintains stability in low-data regimes and achieves continuous improvement, suggesting that sufficiently expressive models can gain incrementally without undergoing transient degradation.

Data Ratio: As shown in Figure 13 and Table 2, increasing the proportion of single-API to multi-API samples from 1:9 leads to a performance trend that first improves and then declines, with the 8:2 ratio achieving the best results under our setting.

This finding is consistent with prior work [15, 22], which shows that combining a larger share of simpler examples with a smaller portion of complex ones can enhance code reasoning and supervised fine-tuning effectiveness. Accordingly, RQ1 adopts the empirically optimal 8:2 ratio.

Table 2. The Impact of Data Ratio on Model Fine-Tuning.

Model	1:9	2:8	3:7	4:6	5:5	6:4	7:3	8:2	9:1
Deepseek-R1-7B	1.85%	3.70%	5.56%	7.41%	7.41%	8.33%	8.33%	9.26%	6.48%
Mistral-7B	5.56%	7.41%	8.33%	9.26%	10.19%	12.96%	14.81%	14.81%	12.04%
Qwen2.5-Coder-7B	9.26%	12.96%	15.74%	19.44%	20.37%	22.22%	23.15%	25.00%	20.37%

Table 3. The Impact of Data Volume on Model Fine-Tuning.

Model	0	1,000	2,000	3,000	4,000	5,000	6,000	7,000	8,000
Deepseek-R1-7B	0%	2.78%	5.56%	3.70%	1.85%	6.48%	6.48%	8.33%	9.26%
Mistral-7B	4.63%	6.48%	6.48%	7.41%	5.56%	7.41%	10.19%	12.04%	14.81%
Qwen2.5-Coder-7B	8.80%	8.80%	9.26%	12.04%	16.67%	18.52%	20.37%	23.15%	25.00%

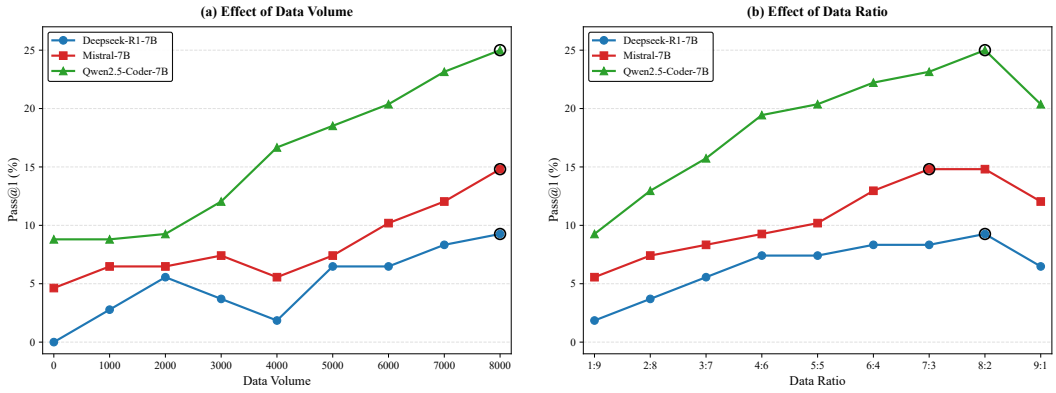


Fig. 13. Impact of Data Scale and Data Ratio on Fine-tuning Performance.

Finding 2: For API-oriented problem-code tuples, larger datasets generally improve fine-tuning performance, and smaller models may exhibit double descent. Performance is highest when the single-API to multi-API ratio is approximately 4:1, suggesting that a training distribution dominated by simpler API interactions with some complex examples supports effective adaptation and generalization.

3.4 RQ3: Distinction and Roles of Single-API and Multi-API Data

Design. To better investigate the impact of single-API and multi-API data, we conducted both quantitative and qualitative experiments. For the quantitative study, all multi-API samples were removed from the original 8,000-sample dataset to assess their effect on fine-tuning performance. Additionally, all single-API samples were also excluded, allowing us to examine the impact of a dataset composed solely of multi-API samples. For the qualitative study, the 1,600 multi-API samples and 6,400 single-API samples were partitioned into three groups, each containing 100 randomly selected single-API and multi-API samples. The sampling process was strictly random to ensure unbiased representativeness.

Investigating the Impact of Removing Multi-API Data on Model Fine-Tuning. We selected the best-performing model, Qwen2.5-Coder-7B, and fine-tuned it on the dataset with multi-API data removed, in order to investigate the impact of multi-API data.

Table 4. Impact of different training data types on fine-tuning performance.

Model	Single-API (6,400)	Multi-API (1,600)	All (8,000)
Qwen2.5-Coder-7B	10.19%	9.26%	25.00%

Investigating the Impact of Removing Single-API Data on Model Fine-Tuning. To ensure consistency, the same models used in the aforementioned experiments were employed.

Investigating the Distribution of Different Types of API Data. To explore the relationship between multi-API and single-API data, we first approach it from a feature-based perspective. By embedding the questions and code in each sample, we obtain high-dimensional representations for both single-API and multi-API data. Principal Component Analysis (PCA) is then applied to reduce the dimensionality, resulting in the visualization shown in Figure 14.

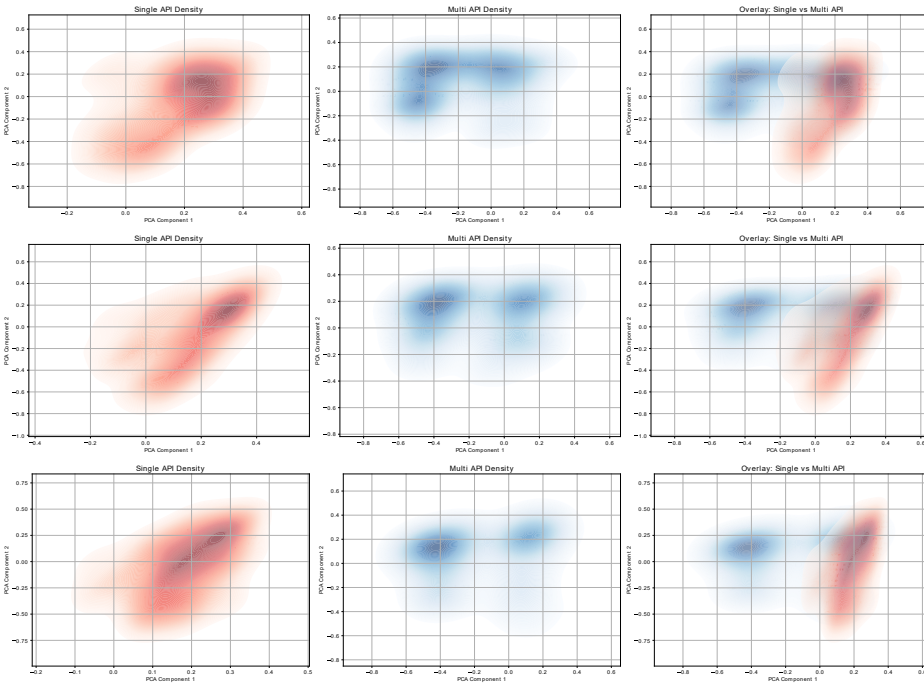


Fig. 14. PCA visualizations for three groups (100 features each). Red/Blue points denote Single/Multi-API samples. The rightmost plot shows the combined distribution of both data types.

Results. Table 4 shows that removing multi-API data reduces the pass@1 score from 25.00% to 10.19%, indicating a substantial performance drop. This highlights the critical role of multi-API data in fine-tuning. Interestingly, when single-API data is removed, the performance decreases to 9.26%, which is even lower than the 10.19% observed when multi-API data is excluded. This underscores the fundamental importance of single-API data, which is further illustrated by the qualitative results below. In the figure 14, darker regions indicate areas where features are densely concentrated, while lighter regions correspond to sparser distributions. From the overlapping plots, it can be observed that multi-API data complements regions underrepresented by single-API data, as areas not covered by red are covered by blue. These observations provide qualitative evidence that multi-API data helps to fill the gaps left by single-API data

Finding 3: Multi-API data can, to some extent, compensate for the knowledge deficiencies inherent in single-API data.

3.5 RQ4: Component Effectiveness

Design. To evaluate the contribution of each component in APIKG4Syn, we replace the MCTS retrieval strategy with Random and Frequency-based alternatives, and substitute the Knowledge Graph with plain text.

MCTS Strategy Replacement. We assess MCTS by replacing it with: (1) a Random strategy that selects non-module nodes from the KG to construct multi-API data, and (2) a Frequency-based strategy that prioritizes APIs with higher in-degree and out-degree, reflecting stronger parameter or return-value connections.

RAG-Based Direct Code Generation Without Fine-Tuning. Since MCTS relies on graph structures, we replace the knowledge graph with the original textual corpus and apply RAG on OHBen without fine-tuning, directly retrieving information to generate code for each query, thereby evaluating the effectiveness of structured knowledge compared to unstructured text.

Table 5. Impact of different component strategies on model performance.

Method	Search Strategy	Fine-tuning	Pass@1 (%)
Plain Text + RAG	None	✗	9.26
APIKG4Syn (Variants)	Random	✓	20.37
	Frequency	✓	22.22
APIKG4Syn	MCTS	✓	25.00

Results. As shown in Table 5, replacing the Knowledge Graph with a plain-text RAG approach achieves only 9.26% Pass@1, slightly above direct generation (8.33%), indicating limited benefit. This is mainly due to the large, definition-heavy source corpus, which hinders accurate retrieval for complex multi-API scenarios. Replacing MCTS with random and frequency-based strategies results in Pass@1 scores of 20.37% and 22.22%, respectively, both substantially lower than the full APIKG4Syn method. The random strategy often combines irrelevant APIs, while the frequency-based strategy captures parameter-coupled APIs but misses semantically related combinations without explicit parameter links. These results demonstrate that the LLM-derived UE value better captures scenario-level API similarity, enabling APIKG4Syn to construct higher-quality multi-API training data.

Finding 4: In data-scarce settings, RAG cannot replace synthetic data fine-tuning. Furthermore, UE-guided MCTS significantly outperforms baselines in generating API combinations, validating APIKG4Syn’s effectiveness.

4 Threats to Validity

Internal Validity. A key threat to internal validity is the potential overlap between the training data and the benchmark, which could inflate model performance in RQ2 experiments on different data volumes. To address this, we used a fuzzy text-matching deduplication process (described in Section 2.3) to remove similar samples and maintain internal validity.

External Validity. A major threat to external validity concerns the potential influence of model pretraining on fine-tuning, as models released later may have greater exposure to low-resource data over time. However, RQ1 findings indicate that although Deepseek-R1-7B and Qwen2.5-Coder-7B

were released around the same time, Qwen2.5-Coder-7B achieves better performance. Similarly, Mistral-7B, released earlier than Deepseek-R1-7B, also demonstrates superior performance. These observations indirectly suggest that the timing and data of model pretraining have limited impact in low-resource scenarios.

Threats to Generalizability. Our experiments and data generation were conducted only in low-resource scenarios on HarmonyOS, which may limit the generalizability of our findings. To test the broader applicability of the APIKG4Syn approach, we plan to extend our deployment to other low-resource environments in the future to validate the generalizability of our method.

5 Related Work

5.1 Data Synthesis for Code LLMs

In recent years, LLMs have shown remarkable capabilities in software engineering tasks such as code generation [8–10, 17, 31], code completion [12, 55], fault localization and repair [37, 51, 54], and code summarization [41, 50]. The performance of modern LLMs is largely dependent on the quality of the training data. Given that manually constructing high-quality and large-scale instruction datasets is both time-consuming and costly, leveraging LLMs to automatically generate synthetic data has become a key enabling technology driving progress in this field.

Recent work explores synthesizing training data for code generation to improve LLMs without extensive manual annotation. Ma et al. [30] propose UnitCoder, a scalable pipeline that generates verifiable programs guided by unit tests, improving code LLMs via synthetic training data. Majumdar et al. [32] introduce Genetic Instruct, an evolutionary method to scale instruction-tuning data for code generation by bootstrapping LLMs from seed samples. Shao et al. [40] present Case2Code, a method to synthesize input-output behavior-based code generation data, enabling effective model training without human annotations. Gupta et al. [13] propose TarGEN, a self-correcting synthetic data generation framework using multi-step prompting for diverse NLP tasks, including code.

Existing approaches to data synthesis often rely on simple code snippets, fixed templates, or single-path API usage examples. In contrast, our work introduces subgraph-based prompts that explicitly encode element-level invocation and dependency relationships, leading to more realistic and semantically coherent data generation. Moreover, the adoption of a multi-path sampling strategy significantly enhances the diversity of the synthesized data.

5.2 Domain-Specific and Low-Resource Code Generation

Recent advances in code generation have increasingly addressed challenges associated with domain-specific languages (DSLs) and low-resource programming languages, such as limited data availability, unique syntax, and semantic constraints. Such as Bridge-Coder [57] and SPEAC [34], enhance code generation for low-resource and DSLs by generating synthetic bridging data or intermediate representations that improve alignment between natural language and target code syntax. These techniques help mitigate data scarcity and improve syntactic and semantic correctness. Giagnorio et al. [11] further investigate fine-tuning strategies across different model sizes, showing that careful pretraining and adaptation significantly impact performance on languages like R and Racket. DreamCoder [6] takes a program synthesis approach, using a wake-sleep learning cycle to construct and expand DSLs, enabling the generation of structured, domain-relevant programs from natural language. AceCoder [21] explores prompt engineering and retrieval-based augmentation by incorporating in-context examples selected from a code corpus, leading to substantial improvements in generation accuracy.

Substantial progress has been made in the ArkTS domain, with studies advancing tasks like code translation and analysis. Liu et al. [28] propose a Java-to-ArkTS converter that uses LLMs and the

ThreadBridge library for shared-memory concurrency, enabling the migration of concurrent Java programs. Chen et al. [3] introduce ArkAnalyzer, the first static analysis framework for ArkTS, capable of detecting performance bottlenecks and vulnerabilities. However, research on code generation for low-resource frameworks like ArkTS is still limited. Zhou et al. [59] use an expert-curated knowledge base to translate TS/JS into ArkTS, while Liu et al. [27] apply template-constrained prompting for Java-to-ArkTS conversion. These methods often miss API usage modeling and lead to redundant re-implementations instead of reusing ArkTS APIs. In contrast, our approach generates high-quality training data from standard API documentation, with multi-API compositions and complex relationships, addressing data scarcity in low-resource scenarios. Unlike rigid prompt-based methods, our solution adapts to any API documentation without manual input, making it applicable to various API ecosystems.

5.3 Knowledge Graph in Software Engineering

Knowledge Graphs are widely used in software engineering to improve efficiency. In software maintenance, they help identify code dependencies and issues, making debugging and refactoring easier [14, 53]. In code analysis, they provide a structured view of software components, assisting with static and dynamic analysis [18, 38]. Knowledge Graphs are also used in automation, allowing intelligent systems to reason about data relationships, supporting tasks like code generation, repair [4], and testing. These applications highlight the potential of Knowledge Graphs in improving the software engineering life cycle.

In the domain of software engineering, researchers have established knowledge graphs for diverse objectives, encompassing domain terminology [45, 46], programming cases [24], bug tracking [42, 47], API concepts [19, 25, 29, 52], API documentation [26, 36]. The knowledge graph we have constructed focuses on the HarmonyOS API documentation, with an emphasis on capturing the hierarchical and semantic relationships between different APIs (such as functional descriptions, input parameters, and return values). By structuring the real-world connections between APIs, the graph brings the representation closer to actual application scenarios, thus providing more realistic and practical support for generating training data for both single-API and multi-API use cases.

6 Conclusion

This paper introduces a method to generate API-oriented question-code datasets for low-resource scenarios using API knowledge graphs. Previous studies show that while LLMs are strong in reasoning about code logic, they often struggle with the syntax and API usage of low-resource languages. Our approach, APIKG4SYN, doesn't require executable code and generates high-quality datasets purely from API documentation. This method is practical for companies or proprietary frameworks that want to fine-tune private LLMs without large-scale training data. Our experiment with HarmonyOS shows that a smaller model, Qwen2.5-Coder, fine-tuned with API data, outperforms larger models like GPT-4o without fine-tuning, highlighting the benefits of API-oriented data for improving API knowledge in low-resource settings.

7 Data Availability

To facilitate the replication study, we have released our data and code at : <https://github.com/SYSUSELab/APIKG4SYN>.

Acknowledgments

This work is supported by National Natural Science Foundation of China (No. 92582117, No. 62402113, No. 92582202), CCF-Huawei Populus Grove Fund CCF-HuaweiSE2025005, and GMCC-SYSU Joint Lab for Smart Applications.

References

- [1] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-Time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47, 3 (2002), 235–256. doi:10.1023/A:1013689704352
- [2] Yupeng Chang, Chenlu Guo, Yi Chang, and Yuan Wu. 2025. LoRA-MGPO: Mitigating Double Descent in Low-Rank Adaptation via Momentum-Guided Perturbation Optimization. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. 648–659.
- [3] Haonan Chen, Daihang Chen, Yizhuo Yang, Lingyun Xu, Liang Gao, Mingyi Zhou, Chunming Hu, and Li Li. 2025. ArkAnalyzer: The Static Analysis Framework for OpenHarmony. arXiv:2501.05798 [cs.SE] <https://arxiv.org/abs/2501.05798>
- [4] Zimin Chen, Yue Pan, Siyu Lu, Jiayi Xu, Claire Le Goues, Martin Monperrus, and He Ye. 2025. Prometheus: Unified Knowledge Graphs for Issue Resolution in Multilingual Codebases. arXiv:2507.19942 [cs.SE] <https://arxiv.org/abs/2507.19942>
- [5] Yifeng Di and Tianyi Zhang. 2025. Enhancing Code Generation via Bidirectional Comment-Level Mutual Grounding. arXiv:2505.07768 [cs.SE] <https://arxiv.org/abs/2505.07768>
- [6] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sable-Meyer, Luc Cary, Lucas Morales, Luke Hewitt, Armando Solar-Lezama, and Joshua B. Tenenbaum. 2020. DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning. arXiv:2006.08381 [cs.AI] <https://arxiv.org/abs/2006.08381>
- [7] DeepSeek-AI et al. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] <https://arxiv.org/abs/2412.19437>
- [8] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [9] Tom B et al. 2020. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs.CL] <https://arxiv.org/abs/2005.14165>
- [10] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139
- [11] Alessandro Giagnorio, Alberto Martin-Lopez, and Gabriele Bavota. 2025. Enhancing Code Generation for Low-Resource Languages: No Silver Bullet. arXiv:2501.19085 [cs.SE] <https://arxiv.org/abs/2501.19085>
- [12] Daya Guo, Canwen Xu, Nan Duan, Jian Yin, and Julian McAuley. 2023. LongCoder: a long-range pre-trained language model for code completion. In *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, USA) (ICML '23). JMLR.org, Article 486, 10 pages.
- [13] Himanshu Gupta, Kevin Scaria, Ujjwala Anantheswaran, Shreyas Verma, Mihir Parmar, Saurabh Arjun Sawant, Chitta Baral, and Swaroop Mishra. 2024. TarGEN: Targeted Data Generation with Large Language Models. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=gpgMRWgv9Q>
- [14] Zhuobing Han, Xiaohong Li, Hongtao Liu, Zhenchang Xing, and Zhiyong Feng. 2018. DeepWeak: Reasoning common software weaknesses via knowledge graph embedding. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 456–466. doi:10.1109/SANER.2018.8330232
- [15] Muyu He, Muhammad Ali Shafique, Anand Kumar, Tsach Mackey, and Nazneen Rajani. 2025. The Valley of Code Reasoning: Scaling Knowledge Distillation of Large Language Models. arXiv:2510.06101 [cs.CL] <https://arxiv.org/abs/2510.06101>
- [16] Xue Jiang, Yihong Dong, Yongding Tao, Huanyu Liu, Zhi Jin, Wenpin Jiao, and Ge Li. 2025. ROCODE: Integrating Backtracking Mechanism and Program Analysis in Large Language Models for Code Generation. arXiv:2411.07112 [cs.SE] <https://arxiv.org/abs/2411.07112>
- [17] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-Planning Code Generation with Large Language Models. *ACM Trans. Softw. Eng. Methodol.* 33, 7, Article 182 (Sept. 2024), 30 pages. doi:10.1145/3672456
- [18] Saravanan Krishnan, Amith Singhee, Keerthi Narayan Raghunath, Alex Mathai, Atul Kumar, and David Wenk. 2025. Incremental Analysis of Legacy Applications Using Knowledge Graphs for Application Modernization. arXiv:2505.06885 [cs.SE] <https://arxiv.org/abs/2505.06885>
- [19] Hongwei Li, Sirui Li, Jiamou Sun, Zhenchang Xing, Xin Peng, Mingwei Liu, and Xuejiao Zhao. 2018. Improving API Caveats Accessibility by Mining API Caveats Knowledge Graph. In *2018 IEEE International Conference on Software Maintenance and Evolution, ICSME 2018, Madrid, Spain, September 23-29, 2018*. IEEE Computer Society, 183–193. doi:10.1109/ICSME.2018.00028
- [20] Jia Li, Yongmin Li, Ge Li, Zhi Jin, Yiyang Hao, and Xing Hu. 2023. SKCODER: A Sketch-Based Approach for Automatic Code Generation. In *Proceedings of the 45th International Conference on Software Engineering* (Melbourne, Victoria, Australia) (ICSE '23). IEEE Press, 2124–2135. doi:10.1109/ICSE48619.2023.00179
- [21] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2023. AceCoder: Utilizing Existing Code to Enhance Code Generation. arXiv:2303.17780 [cs.SE] <https://arxiv.org/abs/2303.17780>

- [22] Yuan Li, Zhengzhong Liu, and Eric Xing. 2025. Data Mixing Optimization for Supervised Fine-Tuning of Large Language Models. arXiv:2508.11953 [cs.AI] <https://arxiv.org/abs/2508.11953>
- [23] Feng Lin, Dong Jae Kim, Tse-Husn, and Chen. 2024. SOEN-101: Code Generation by Emulating Software Process Models Using Large Language Model Agents. arXiv:2403.15852 [cs.SE] <https://arxiv.org/abs/2403.15852>
- [24] Mingwei Liu, Xin Peng, Andrian Marcus, Christoph Treude, Jiazhan Xie, Huanjun Xu, and Yanjun Yang. 2022. How to formulate specific how-to questions in software development?. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14–18, 2022*. ACM, 306–318. doi:10.1145/3540250.3549160
- [25] Mingwei Liu, Xin Peng, Andrian Marcus, Zhenchang Xing, Wenkai Xie, Shuangshuang Xing, and Yang Liu. 2019. Generating query-specific class API summaries. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26–30, 2019*. ACM, 120–130. doi:10.1145/3338906.3338971
- [26] Mingwei Liu, Xin Peng, Xiujie Meng, Huanjun Xu, Shuangshuang Xing, Xin Wang, Yang Liu, and Gang Lv. 2020. Source Code based On-demand Class Documentation Generation. In *IEEE International Conference on Software Maintenance and Evolution, ICSME 2020, Adelaide, Australia, September 28 - October 2, 2020*. IEEE, 864–865. doi:10.1109/ICSME46990.2020.00114
- [27] Runlin Liu, Yuhang Lin, Yunge Hu, Zhe Zhang, and Xiang Gao. 2024. Llm-based java concurrent program to arkts converter. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 2403–2406.
- [28] Runlin Liu, Yuhang Lin, Yunge Hu, Zhe Zhang, and Xiang Gao. 2024. LLM-Based Java Concurrent Program to ArkTS Converter. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)* (Sacramento, CA, USA). Association for Computing Machinery (ACM), 2403–2406. doi:10.1145/3691620.3695362
- [29] Yang Liu, Mingwei Liu, Xin Peng, Christoph Treude, Zhenchang Xing, and Xiaoxin Zhang. 2020. Generating Concept based API Element Comparison Using a Knowledge Graph. In *35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020, Melbourne, Australia, September 21–25, 2020*. IEEE, 834–845. doi:10.1145/3324884.3416628
- [30] Yichuan Ma, Yunfan Shao, Peiji Li, Demin Song, Qipeng Guo, Linyang Li, Xipeng Qiu, and Kai Chen. 2025. UnitCoder: Scalable Iterative Code Synthesis with Unit Test Guidance. arXiv:2502.11460 [cs.CL] <https://arxiv.org/abs/2502.11460>
- [31] Yingwei Ma, Yue Yu, Shanshan Li, Yu Jiang, Yong Guo, Yuanliang Zhang, Yutao Xie, and Xiangke Liao. 2023. Bridging Code Semantic and LLMs: Semantic Chain-of-Thought Prompting for Code Generation. arXiv:2310.10698 [cs.CL] <https://arxiv.org/abs/2310.10698>
- [32] Somshubra Majumdar, Wahid Noroozi, Mehrzad Samadi, Sean Narenthiran, Aleksander Ficek, Wasi Uddin Ahmad, Jocelyn Huang, Jagadeesh Balam, and Boris Ginsburg. 2025. Genetic Instruct: Scaling up Synthetic Generation of Coding Instructions for Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, Georg Rehm and Yunyao Li (Eds.). Association for Computational Linguistics, Vienna, Austria, 208–221. doi:10.18653/v1/2025.acl-industry.16
- [33] Nicholas Metropolis and Stanislaw Ulam. 1949. The Monte Carlo method. *J. Amer. Statist. Assoc.* 44, 247 (1949), 335–341. doi:10.1080/01621459.1949.10483310
- [34] Federico Mora, Justin Wong, Haley Lepe, Sahil Bhatia, Karim Elmaaroufi, George Varghese, Joseph E. Gonzalez, Elizabeth Polgreen, and Sanjit A. Seshia. 2024. Synthetic Programming Elicitation for Text-to-Code in Very Low-Resource Programming and Formal Languages. arXiv:2406.03636 [cs.PL] <https://arxiv.org/abs/2406.03636>
- [35] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, 1–13. doi:10.1145/3597503.3639226
- [36] Xin Peng, Yifan Zhao, Mingwei Liu, Fengyi Zhang, Yang Liu, Xin Wang, and Zhenchang Xing. 2018. Automatic Generation of API Documentations for Open-Source Projects. In *IEEE Third International Workshop on Dynamic Software Documentation, DySDoc@ICSME 2018, Madrid, Spain, September 25, 2018*. IEEE, 7–8. doi:10.1109/DySDoc3.2018.00010
- [37] Asif Mohammed Samir and Mohammad Masudur Rahman. 2025. Improved IR-based Bug Localization with Intelligent Relevance Feedback. arXiv:2501.10542 [cs.SE] <https://arxiv.org/abs/2501.10542>
- [38] Dennis Schiese, Aleksandr Perevalov, and Andreas Both. 2024. Towards LLM-generated explanations for component-based knowledge graph question answering systems. (9 2024). doi:10.6084/m9.figshare.27079687.v2
- [39] C. E. Shannon. 1948. A mathematical theory of communication. *The Bell System Technical Journal* 27, 3 (1948), 379–423. doi:10.1002/j.1538-7305.1948.tb01338.x
- [40] Yunfan Shao, Linyang Li, Yichuan Ma, Peiji Li, Demin Song, Qinyuan Cheng, Shimin Li, Xiaonan Li, Pengyu Wang, Qipeng Guo, Hang Yan, Xipeng Qiu, Xuanjing Huang, and Dahua Lin. 2025. Case2Code: Scalable Synthetic Data for Code Generation. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 11056–11069. <https://aclanthology.org/2025.coling-main.733/>

- [41] Chia-Yi Su, Aakash Bansal, Yu Huang, Toby Jia-Jun Li, and Collin McMillan. 2024. Context-aware Code Summary Generation. arXiv:2408.09006 [cs.SE] <https://arxiv.org/abs/2408.09006>
- [42] Yanqi Su, Zhenchang Xing, Xin Peng, Xin Xia, Chong Wang, Xiwei Xu, and Liming Zhu. 2021. Reducing Bug Triaging Confusion by Learning from Mistakes with a Bug Tossing Knowledge Graph. In *36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021, Melbourne, Australia, November 15-19, 2021*. IEEE, 191–202. doi:10.1109/ASE51524.2021.9678574
- [43] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [44] Zhao Tian, Junjie Chen, and Xiangyu Zhang. 2024. Fixing Large Language Models' Specification Misunderstanding for Better Code Generation. arXiv:2309.16120 [cs.SE] <https://arxiv.org/abs/2309.16120>
- [45] Chong Wang, Xin Peng, Mingwei Liu, Zhenchang Xing, Xuefang Bai, Bing Xie, and Tuo Wang. 2019. A learning-based approach for automatic construction of domain glossary from source code and documentation. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019*. ACM, 97–108. doi:10.1145/3338906.3338963
- [46] Chong Wang, Xin Peng, Zhenchang Xing, Yue Zhang, Mingwei Liu, Rong Luo, and Xiujie Meng. 2023. XCoS: Explainable Code Search based on Query Scoping and Knowledge Graph. *ACM Transactions on Software Engineering and Methodology* (2023).
- [47] Lu Wang, Xiaobing Sun, Jingwei Wang, Yucong Duan, and Bin Li. 2017. Construct bug knowledge graph for bug resolution: poster. In *Proceedings of the 39th International Conference on Software Engineering, ICSE 2017, Buenos Aires, Argentina, May 20-28, 2017 - Companion Volume*. IEEE Computer Society, 189–191. doi:10.1109/ICSE-C.2017.102
- [48] Yunkun Wang, Yue Zhang, Zhen Qin, Chen Zhi, Binhua Li, Fei Huang, Yongbin Li, and Shuiguang Deng. 2025. ExploraCoder: Advancing code generation for multiple unseen APIs via planning and chained exploration. arXiv:2412.05366 [cs.SE] <https://arxiv.org/abs/2412.05366>
- [49] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2024. Magicoder: Empowering Code Generation with OSS-Instruct. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 52632–52657. <https://proceedings.mlr.press/v235/wei24h.html>
- [50] Yang Wu, Yao Wan, Zhaoyang Chu, Wenting Zhao, Ye Liu, Hongyu Zhang, Xuanhua Shi, Hai Jin, and Philip S. Yu. 2025. Can Large Language Models Serve as Evaluators for Code Summarization? *IEEE Transactions on Software Engineering* 51, 12 (2025), 3205–3217. doi:10.1109/TSE.2025.3595283
- [51] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489 [cs.SE] <https://arxiv.org/abs/2407.01489>
- [52] Shuangshuang Xing, Mingwei Liu, and Xin Peng. 2021. Automatic Code Semantic Tag Generation Approach Based on Software Knowledge Graph. *Journal of Software* 33, 11 (2021), 4027–4045.
- [53] Xiangzhe Xu, Guangyu Shen, Zian Su, Siyuan Cheng, Hanxi Guo, Lu Yan, Xuan Chen, Jiasheng Jiang, Xiaolong Jin, Chengpeng Wang, Zhuo Zhang, and Xiangyu Zhang. 2025. ASTRA: Autonomous Spatial-Temporal Red-teaming for AI Software Assistants. arXiv:2508.03936 [cs.CR] <https://arxiv.org/abs/2508.03936>
- [54] Aidan Z. H. Yang, Ruben Martins, Claire Le Goues, and Vincent J. Hellendoorn. 2023. Large Language Models for Test-Free Fault Localization. arXiv:2310.01726 [cs.SE] <https://arxiv.org/abs/2310.01726>
- [55] Dongjun Yu, Xiao Yan, Zhenrui Li, Jipeng Xiao, Haochuan He, Yongda Yu, Hao Zhang, Guoping Rong, and Xiaobo Huang. 2025. SynthCoder: A Synthetical Strategy to Tune LLMs for Code Completion. arXiv:2508.15495 [cs.SE] <https://arxiv.org/abs/2508.15495>
- [56] Daoguang Zan, Bei Chen, Zeqi Lin, Bei Guan, Wang Yongji, and Jian-Guang Lou. 2022. When Language Model Meets Private Library. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 277–288. doi:10.18653/v1/2022.findings-emnlp.21
- [57] Wei Zhang, Yi Zhang, Li Zhu, Qianghui Jia, Feijun Jiang, Hongcheng Guo, Zhoujun Li, and Mengping Zhou. 2024. ADC: Enhancing Function Calling Via Adversarial Datasets and Code Line-Level Feedback. arXiv:2412.17754 [cs.SE] <https://arxiv.org/abs/2412.17754>
- [58] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyuan Luo, Zhangchi Feng, and Yongqiang Ma. 2024. LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. Association for Computational Linguistics, Bangkok, Thailand. <http://arxiv.org/abs/2403.13372>
- [59] Bo Zhou, Jiaqi Shi, Ying Wang, Li Li, Tsz On Li, Hai Yu, and Zhiliang Zhu. 2025. Porting software libraries to OpenHarmony: Transitioning from TypeScript or JavaScript to ArkTS. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1445–1466.

Received 2025-09-12; accepted 2026-03-24